Academic Year: 2025-2026					
Program: Computer Engineering					
Year of Study	Third Year			**Semester**	VI
Lab Name	Data Warehousing And Mining			**Lab Code**	CMCR1603
Class & Div & Batch	TY4 – 04-B	**Roll No**	40	**PRN No.**	23UF17820CM100

Experiment No: 5	
AIM	Implementation of Association Rule Mining Algorithm (Apriori)
Learning Objective	Implement the Apriori algorithm,identifying frequent item sets and deriving actionable association rules
Problem Statement	Generate association rules with a minimum confidence threshold.
Program	

C: > Users > lab603 > dwm.py > ...

```python
from itertools import combinations

# Sample dataset
dataset = [
    ['milk', 'bread', 'butter'],
    ['beer', 'bread'],
    ['milk', 'bread', 'beer', 'butter'],
    ['bread', 'butter'],
    ['milk', 'bread', 'butter']
]

min_support = 0.6  # Minimum support threshold
min_confidence = 0.7  # Minimum confidence threshold

# Step 1: Create initial candidate itemsets (C1)
def create_C1(dataset):
    C1 = set()
    for transaction in dataset:
        for item in transaction:
            C1.add(frozenset([item]))
    return list(C1)

# Step 2: Calculate support
def calculate_support(dataset, candidates):
    support_count = {}
    for candidate in candidates:
        for transaction in dataset:
            if candidate.issubset(transaction):
                support_count[candidate] = support_count.get(candidate, 0) + 1
    total_transactions = len(dataset)
    support = {item: count/total_transactions for item, count in support_count.items()}
    return support

# Step 3: Filter candidates by min support
def filter_candidates(support, min_support):
    return {item for item, sup in support.items() if sup >= min_support}

```

```python
# Step 4: Generate new candidate itemsets from previous frequent itemsets
def apriori_gen(frequent_itemsets, k):
    candidates = set()
    frequent_list = list(frequent_itemsets)
    for i in range(len(frequent_list)):
        for j in range(i+1, len(frequent_list)):
            union_set = frequent_list[i].union(frequent_list[j])
            if len(union_set) == k:
                candidates.add(union_set)
    return candidates

# Step 5: Generate all frequent itemsets
def apriori(dataset, min_support):
    C1 = create_C1(dataset)
    L1 = filter_candidates(calculate_support(dataset, C1), min_support)
    L = [L1]
    k = 2
    while L[k-2]:
        Ck = apriori_gen(L[k-2], k)
        support_Ck = calculate_support(dataset, Ck)
        Lk = filter_candidates(support_Ck, min_support)
        if Lk:
            L.append(Lk)
        else:
            break
        k += 1
    return L

# Step 6: Generate association rules
def generate_rules(L, dataset, min_confidence):
    rules = []
```

```python
    rules = []
    for i in range(1, len(L)):
        for freq_set in L[i]:
            for j in range(1, len(freq_set)):
                for subset in combinations(freq_set, j):
                    antecedent = frozenset(subset)
                    consequent = freq_set - antecedent
                    if consequent:
                        support_freq_set = calculate_support(dataset, [freq_set])[freq_set]
                        support_antecedent = calculate_support(dataset, [antecedent])[antecedent
                        confidence = support_freq_set / support_antecedent
                        if confidence >= min_confidence:
                            rules.append((antecedent, consequent, confidence))
    return rules

# Run Apriori
frequent_itemsets = apriori(dataset, min_support)
print("Frequent Itemsets:")
for level in frequent_itemsets:
    print(level)

rules = generate_rules(frequent_itemsets, dataset, min_confidence)
print("\nAssociation Rules:")
for rule in rules:
    print(f"{set(rule[0])} -> {set(rule[1])}, confidence: {rule[2]:.2f}")
```

Output	PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PS C:\Users\lab603> & 'C:\Users\lab603\AppData\Local\Microsoft\WindowsApps\python3.13.exe' Frequent Itemsets: {frozenset({'milk'}), frozenset({'bread'}), frozenset({'butter'})} {frozenset({'bread', 'milk'}), frozenset({'bread', 'butter'}), frozenset({'butter', 'milk'})} {frozenset({'bread', 'butter', 'milk'})} Association Rules: {'milk'} -> {'bread'}, confidence: 1.00 {'bread'} -> {'butter'}, confidence: 0.80 {'butter'} -> {'bread'}, confidence: 1.00 {'butter'} -> {'milk'}, confidence: 0.75 {'milk'} -> {'butter'}, confidence: 1.00 {'butter'} -> {'bread', 'milk'}, confidence: 0.75 {'milk'} -> {'bread', 'butter'}, confidence: 1.00 {'bread', 'butter'} -> {'milk'}, confidence: 0.75 {'bread', 'milk'} -> {'butter'}, confidence: 1.00 {'butter', 'milk'} -> {'bread'}, confidence: 1.00 PS C:\Users\lab603>
Conclusion	The Apriori algorithm successfully identified frequent itemsets and generated association rules, revealing meaningful relationships in the dataset.